

PROMETHEE II в `internal/domain/selection`

Документ описывает, как именно работает алгоритм PROMETHEE II в этом проекте: какие данные принимает на вход, какие шаги выполняет, какие формулы используются и как устроен конечный ранжированный список треков.

Реализация основана на главе 3 книги

Jason Papathanasiou, Nikolaos Ploskas – Multiple Criteria Decision Aid Methods (стр. 57–76).

1. Назначение

Студент заполнил профиль (оценки, навыки, цели, предпочтения по сертификатам и стилю обучения). Каждый трек обладает набором характеристик (сложность, перспективы трудоустройства, отзывы выпускников, желаемые навыки и т. п.).

Задача – отсортировать треки по тому, насколько они подходят конкретному студенту, при этом учесть **много критериев одновременно** и веса этих критериев.

Это задача **многокритериального принятия решения** (MCDM, Multiple-Criteria Decision Making). PROMETHEE II – один из стандартных алгоритмов её решения, который даёт **полное упорядочение** альтернатив (в отличие от PROMETHEE I, который выдаёт частичный порядок).

2. Высокоуровневая идея PROMETHEE II

> «Насколько трек *a* в среднем превосходит остальные треки и насколько > остальные превосходят его?»

Алгоритм:

1. Каждый трек оценивается по каждому критерию числом из $[0, 1]$, где «больше = лучше».
2. Для каждой **упорядоченной пары** треков (a, b) и каждого критерия считается, насколько a предпочтительнее b по этому критерию (от 0 до 1). Это делает **функция предпочтения** $P_j(d)$.
3. По всем критериям результат агрегируется во **взвешенный индекс предпочтения** $\pi(a, b) \in [0, 1]$.
4. У каждого трека считаются три «потока»:
 - * $\Phi^+(a)$ – **позитивный поток** – насколько a в среднем превосходит остальные.
 - * $\Phi^-(a)$ – **негативный поток** – насколько остальные в среднем превосходят a .
 - * $\Phi(a) = \Phi^+(a) - \Phi^-(a)$ – **чистый поток**.
5. Треки сортируются по Φ убыванию.

Числовое значение $\Phi \in [-1, 1]$:

- * $\Phi = 1$ – трек строго доминирует все остальные по всем критериям;
- * $\Phi = -1$ – наоборот;
- * $\Phi = 0$ – трек «средний» в выборке.

3. Публичные типы

Все типы лежат в `promethee.go`.

`PrometheeInput`

Входные данные одного трека, как они нужны алгоритму (``promethee.go:18``). Заполняется в `use-case`application/student/get_recommendations.go:85``.

Поле	Тип	Смысл
<code>`TrackID`</code> , <code>`TrackName`</code>	<code>`string`</code>	идентификация трека
<code>`ProfessionalGoals`</code>	<code>`[]int`</code>	какие профессии трек готовит
<code>`Employment`</code>	<code>`int 1-10`</code>	перспективы трудоустройства
<code>`AlumniReviews`</code>	<code>`int 1-10`</code>	средний отзыв выпускников
<code>`Difficulty`</code>	<code>`int 1-5`</code>	сложность
<code>`HasCertificates`</code>	<code>`0/1`</code>	выдаются ли сертификаты
<code>`LearningStyle`</code>	<code>`1/2/3`</code>	теория / практика / смешанный
<code>`DesiredTechSkills`</code>	<code>`int 1-10`</code>	какой уровень тех. навыков ожидается
<code>`DesiredMathSkills`</code>	<code>`int 1-10`</code>	какой уровень математики ожидается
<code>`DesiredSoftSkills`</code>	<code>`int 1-10`</code>	какой уровень soft-skills ожидается
<code>`Requirements`</code>	<code>`[]Requirement`</code>	минимальные оценки для зачисления

`StudentData`

Профиль студента (``promethee.go:38``).

Поле	Тип	Смысл
<code>`ProfessionalGoals`</code>	<code>`[]int`</code>	какие профессии нужны студенту
<code>`Grades`</code>	<code>`Grades`</code>	оценки по 9 предметам, шкала 1-5
<code>`Skills`</code>	<code>`Skills`</code>	самооценка 9 навыков, шкала 1-10
<code>`LearningStyle`</code>	<code>`int 1/2/3`</code>	предпочитаемый стиль
<code>`Certificates`</code>	<code>`0/1`</code>	хочет ли сертификаты

`TrackScore`

Результат для одного трека (``promethee.go:8``).

Поле	Смысл
<code>`Score`</code>	чистый поток $\Phi \in [-1, 1]$
<code>`Rank`</code>	позиция после сортировки, начиная с 1
<code>`CriteriaScores`</code>	строка матрицы решений – значения всех критериев в $[0, 1]$
<code>`PositiveFlow`</code>	Φ^+
<code>`NegativeFlow`</code>	Φ^-

`CriteriaWeights` и `DefaultWeights`

Веса критериев (``weights.go``). Веса не обязаны суммироваться в 1 – алгоритм нормализует их сам (см. ниже шаг 5).

`CriterionParams` и шесть функций предпочтений

``promethee.go:79``. Описывает, какую обобщённую функцию предпочтения использовать для одного критерия и с какими параметрами.

```
``go
type PreferenceFunctionType int
const (
    UsualCriterion      = 1
    UShapeCriterion     = 2
    VShapeCriterion     = 3
```

```

LevelCriterion      = 4
LinearCriterion     = 5 // V-shape with indifference
GaussianCriterion   = 6
)

type CriterionParams struct {
    Type PreferenceFunctionType
    Q     float64 // порог безразличия
    P     float64 // порог строгого предпочтения
    S     float64 // ширина гауссианы
}

```

Метод `CriterionParams.preference(d)` возвращает значение функции предпочтения от отклонения `d = gj(a) - gj(b)`.

Шесть функций (см. рис. 3.1 в книге, стр. 60)

При $d \leq 0$ все функции возвращают 0 (если `a` не лучше `b` по этому критерию, то и предпочтения нет). Различаются только в области $d > 0$:

Тип	Формула	Параметры	Когда применять
1. Usual	$P = 1$	$P = 1$	бинарные критерии (0/1): любая разница уже = полное предпочтение
2. U-shape	$P = 1$ если $d > q$, иначе 0	q	дискретное «не важно / очень важно»
3. V-shape	$P = d/p$, обрезано до 1	p	непрерывные критерии, простой линейный рост от 0 до p
4. Level	$P = 0; 0.5; 1$ ступенями	p, q	три уровня предпочтения
5. Linear (V с индифферентностью)	$P = (d-q)/(p-q)$ между q и p , 0 ниже, 1 выше	p, q	непрерывные критерии с явной зоной безразличия
6. Gaussian	$P = 1 - \exp(-d^2/(2s^2))$	s	плавный «сигмоидный» рост

Что используется по умолчанию

`DefaultCriterionParams` (`promethee.go:142`) задаёт:

- * **V-shape** с $P = 1.0$ для непрерывных критериев в $[0, 1]$. Линейный рост предпочтения от полного равенства до максимально возможной разницы.
- * **Usual** для бинарных критериев `certificates` и `learning_style` (значения 0 или 1).

Это даёт разумную «среднюю» поведенческую модель без необходимости руками настраивать пороги безразличия. Если хочется тонко настроить, есть конструктор `NewPrometheeCalculatorWithParams(weights, params)` (`promethee.go:171`).

4. Алгоритм пошагово

Точка входа – `PrometheeCalculator.CalculateScores` (`promethee.go:178`).

Шаг 1. Фильтрация по жёстким требованиям

Метод `meetsRequirements` (`promethee.go:280`):

- * По каждому предмету в `track.Requirements` оценка студента должна быть не ниже минимальной (если ниже – трек выбрасывается).
- * Хотя бы одна профессиональная цель студента должна совпадать с целями трека (если в треке вообще есть цели). Если пересечение пустое – трек выбрасывается.

После этого шага остаётся список ****кандидатов**** – только они участвуют в PROMETHEE.

> Это важный момент: PROMETHEE сам по себе не отсеивает «плохие» альтернативы,
> он только их ранжирует. Жёсткие требования зачисления – это допроцессорный
> фильтр.

Шаг 2. Построение матрицы решений

Для каждого кандидата ``evaluateCriteria`` (``promethee.go:243``) вычисляет ****девять значений критериев****, нормализованных в ``[0, 1]`` так, чтобы «больше = л лучше».

Критерий (ключ в map)	Что значит	Как считается
<code>`professional_goals`</code>	доля целей трека, совпадающих с целями студента	<code>`len(пересечение) / len(track.Goals)`</code> (<code>`promethee.go:307`</code>)
<code>`employment`</code>	трудоустройство	<code>`track.Employment / 10`</code>
<code>`alumni_reviews`</code>	отзывы выпускников	<code>`track.AlumniReviews / 10`</code>
<code>`difficulty`</code>	соответствие сложности трека уровню студента	<code>`1 - ltrack.Difficulty - recommended / 4`</code> , где <code>recommended = 1/2/4</code> в зависимости от среднего балла (<code>`promethee.go:323`</code>)
<code>`certificates`</code>	устраивает ли студента наличие сертификатов	<code>`0`</code> , если студент хочет, а трек не выдаёт; иначе <code>`1`</code> (<code>`promethee.go:347`</code>)
<code>`learning_style`</code>	совпадение стиля обучения	<code>`1`</code> , если совпадает, иначе <code>`0`</code>
<code>`desired_tech_skills`</code>	покрывает ли студент ожидаемый уровень техн. навыков	<code>`min(1, средний_тех_навык / track.DesiredTechSkills)`</code> (<code>`promethee.go:368`</code>)
<code>`desired_math_skills`</code>	то же для математики	средний по <code>`AIG`</code> , <code>`MathAnalysis`</code> , <code>`DiscreteMath`</code> , <code>`MachineLearning`</code>
<code>`desired_soft_skills`</code>	то же для soft-skills	средний по <code>`PublicSpeaking`</code> , <code>`Analytics`</code> , <code>`ForeignLanguage`</code>

В терминах PROMETHEE это и есть «оценки альтернатив по критериям» – ``g_j(a)`` для трека ``a`` и критерия ``j``. Все девять значений лежат в ``[0, 1]`` и все «to maximize».

Шаг 3. Подготовка весов

``weightMap()`` (``promethee.go:267``) возвращает таблицу «имя критерия → вес».
``totalWeight`` – сумма всех весов. Алгоритм будет нормализовать через деление на эту сумму, поэтому абсолютные значения весов в ``CriteriaWeights`` не важны – важны только их ****отношения****.

Шаг 4. Попарные индексы предпочтения

В ``CalculateScores`` (``promethee.go:218``) построена квадратная матрица ``pi[i][j] = π(i, j)``.

Для каждой упорядоченной пары ``(i, j)``, где ``i ≠ j``:

...

для каждого критерия `k` в `criterionOrder`:

```
d = candidates[i].criteria[k] - candidates[j].criteria[k]
Pjk = preferenceFunction[k](d)
sum += w[k] * Pjk
```

```
pi[i][j] = sum / totalWeight
```

Несколько важных деталей:

- * Используется ****детерминированный порядок**** критериев ``criterionOrder`` (``promethee.go:130``). Map в Go обходится в случайном порядке – без явного списка ключей результат был бы недетерминированным из-за накопления ошибки с плавающей точкой.
- * Деление на ``totalWeight`` даёт $\pi \in [0, 1]$. Это ****не**** среднее предпочтение, это взвешенное предпочтение, нормированное так, чтобы суммы весов не смещали шкалу.
- * Если $d \leq 0$ (трек ``i`` не лучше ``j`` по критерию ``k``), то $P_k(d) = 0$. То есть в $\pi(i, j)$ каждый критерий вкладывается только тогда, когда ``i`` по нему действительно лучше.

Шаг 5. Потоки Φ^+ , Φ^- , Φ

После заполнения матрицы ``pi[][]`` в цикле ``i = 0..n-1`` (``promethee.go:246``):

```

$$\begin{aligned}\Phi^+(i) &= \left( \sum_{j \neq i} p_i[i][j], j \neq i \right) / (n - 1) \\ \Phi^-(i) &= \left( \sum_{j \neq i} p_i[j][i], j \neq i \right) / (n - 1) \\ \Phi(i) &= \Phi^+(i) - \Phi^-(i)\end{aligned}$$

- \*  $\Phi^+(i)$  – насколько ``i`` в среднем превосходит остальные.
- \*  $\Phi^-(i)$  – насколько остальные в среднем превосходят ``i``.
- \*  $\Phi(i)$  – итоговый ранжирующий показатель.

Особый случай: если после фильтрации остался **\*\*один трек\*\*** (``n == 1``), то  $n - 1 = 0$  и потоки оставляются нулями. Трек просто получает ``Rank = 1`` и ``Score = 0``.

### Шаг 6. Сортировка и присвоение рангов

```go

```
sort.SliceStable(scores, func(i, j int) bool {
    return scores[i].Score > scores[j].Score
})
for i := range scores {
    scores[i].Rank = i + 1
}
```

```

- \* Сортировка по  $\Phi$  убыванию.
- \* ``SliceStable`` сохраняет порядок треков с одинаковым  $\Phi$  – это удобно, чтобы при равенстве выводилось то, что пришло раньше из БД.
- \* Ранги присваиваются последовательно ``1..n`` (если  $\Phi$  равны, ранги всё равно разные – это решение по простоте; в строгом PROMETHEE такие альтернативы должны называться «индифферентными»).

---

## 5. Маленький пример «на пальцах»

Допустим, после фильтра остались два трека ``A`` и ``B``, веса все равны 1, и для упрощения только два критерия: ``employment`` (V-shape ``P=1``) и ``learning_style`` (Usual).

Значения критериев:

	employment	learning_style
A	0.8	1.0
B	0.5	0.0

#### Считаем  $\pi(A, B)$ :

- \* employment:  $d = 0.8 - 0.5 = 0.3$ . V-shape с  $P=1$ :  $P = 0.3$ . Вклад:  $1 \cdot 0.3 = 0.3$ .
- \* learning\_style:  $d = 1.0 - 0.0 = 1.0$ . Usual:  $P = 1$ . Вклад:  $1 \cdot 1 = 1$ .
- \*  $\pi(A, B) = (0.3 + 1) / 2 = 0.65$ .

#### Считаем  $\pi(B, A)$ :

- \* employment:  $d = -0.3$ ,  $P = 0$ .
- \* learning\_style:  $d = -1.0$ ,  $P = 0$ .
- \*  $\pi(B, A) = 0$ .

#### Потоки ( $n = 2$ ,  $n - 1 = 1$ ):

- \*  $\Phi^+(A) = \pi(A, B) = 0.65$ ,  $\Phi^-(A) = \pi(B, A) = 0$ ,  $\Phi(A) = 0.65$ .
- \*  $\Phi^+(B) = \pi(B, A) = 0$ ,  $\Phi^-(B) = \pi(A, B) = 0.65$ ,  $\Phi(B) = -0.65$ .

Финальный порядок:  $A$  (Rank=1, Score=0.65),  $B$  (Rank=2, Score=-0.65).

Симметрия  $\Phi(A) = -\Phi(B)$  – общее свойство, когда альтернатив всего две.

---

## 6. Где это вызывается

Цепочка вызова со стороны API:

```
...
GET /api/student/recommendations
 → handlers.StudentHandler.GetRecommendations
 → application/student/GetRecommendationsUseCase.Execute
 → загружает PreferencesRepository.FindByUserID(...)
 → загружает TrackRepository.FindAll(...)
 → строит StudentData и []PrometheeInput
 → calculator := NewPrometheeCalculator(DefaultWeights())
 → scores := calculator.CalculateScores(inputs, student)
 → возвращает GetRecommendationsOutput{Recommendations: scores}
...
```

Всё, что выше PROMETHEE – это адаптер между БД-моделями и доменными типами селекшна. Сам алгоритм (`internal/domain/selection`) ни от чего не зависит – это чистая доменная логика.

---

## 7. Как изменять алгоритм

Что хочется поменять	Где
Поменять веса критериев	`weights.go:DefaultWeights` или передавать кастомный `CriteriaWeights` в `NewPrometheeCalculator`
Поменять функции предпочтения по умолчанию	`DefaultCriterionParams` (`promethee.go:142`)
Использовать кастомные функции предпочтения	`NewPrometheeCalculatorWithParams(weights, params)`
Поменять формулу для одного критерия (например, как считается «соответствие сложности»)	соответствующий `calc*` метод (`calcDifficultyMatch`, `calcTechSkillMatch`, ...)
Добавить новый критерий	(1) поле в `PrometheeInput`, (2) поле в `CriteriaWeights`, (3) ключ в `criterionOrder`, (4) ключ в `weightMap`, (5) запись в `evalua`

teCriteria`, (6) запись в `DefaultCriterionParams` |  
| Учесть «to minimize» критерии (например, цена) | в `evaluateCriteria` инвертировать значение: `1 - x\_normalized`. Тогда смысл «больше = лучше» сохранится, и алгоритм трогать не нужно |

---

## ## 8. Ограничения текущей реализации

- \* **Все критерии трактуются как «to maximize»**. Если когда-то появится критерий типа «стоимость» (где меньше = лучше), его нужно либо инвертировать в `evaluateCriteria`, либо расширить `CriterionParams` флагом.
- \* **Сложность  $O(n^2 \cdot m)$** , где `n` – число треков-кандидатов, `m` – число критериев. На текущих масштабах (десятки треков, 9 критериев) это незаметно, но при росте до тысяч треков матрицу `pi` стоит будет считать лениво.
- \* **Ранги при равенстве  $\Phi$  всё равно разные** – это выбор простоты, а не следование канону PROMETHEE. В каноне нужно либо пометить индифферентность, либо давать одинаковый ранг.
- \* **Только PROMETHEE II**. PROMETHEE I (частичный порядок с «несравнимыми» альтернативами) не реализован – он не нужен фронту, который ожидает плоский упорядоченный список.